



Manual do Programador - Gestão de Sócios

Guia técnico da arquitetura atual, edição manual, testes, Supabase, Git e publicação da área de Sócios.

Ramo	Gestão de Sócios
Destinatários	Programadores e responsáveis técnicos
Repositório	https://github.com/MenteMovimento/central-mente-movimento
Site	https://central-mente-movimento.vercel.app
Atualização	16/07/2026

1. Visão geral da Central

A Central MenteMovimento junta Sócios, Utentes, Cibersegurança e Atividades num único projeto publicado na Vercel e ligado a um único projeto Supabase. A autenticação é centralizada; os dados continuam separados por tabelas e contexto funcional.

- Repositório oficial: <https://github.com/MenteMovimento/central-mente-movimento>
- Site de produção: <https://central-mente-movimento.vercel.app>
- Branch principal: main.
- Build de produção: `npm run build`.
- Diretório publicado pela Vercel: `public`.
- Não publicar ficheiros `.env`, bases SQLite locais, anexos reais, exports com dados pessoais ou chaves privadas.

2. Variáveis e segurança

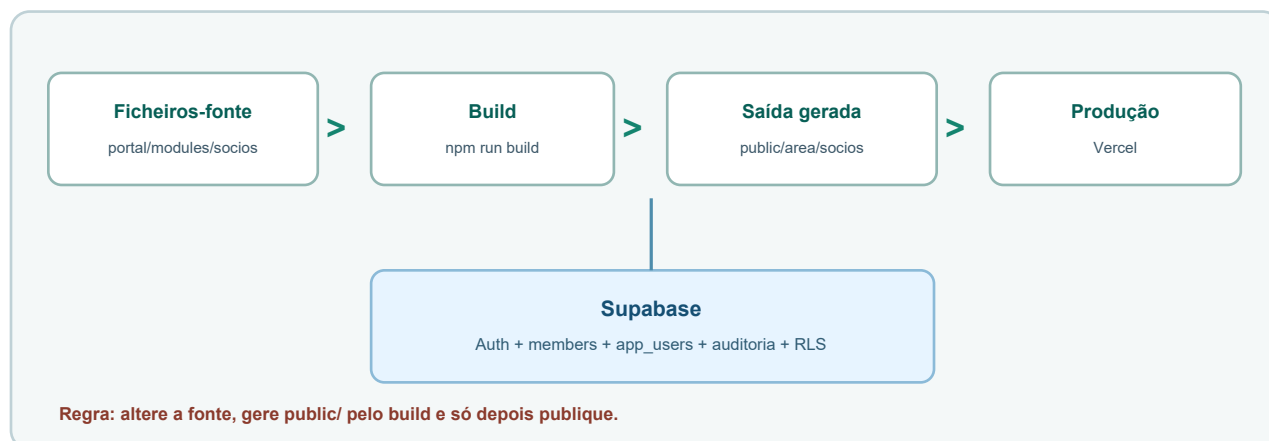
- `SUPABASE_URL` e `SUPABASE_ANON_KEY` podem ser usadas pelo frontend.
- `SUPABASE_SERVICE_ROLE_KEY` é secreta e deve ficar apenas em Environment Variables da Vercel ou ambiente local protegido.
- `SUPABASE_SECRET_KEY` é usada pelo backend Python de Utentes quando aplicável.
- Depois de mudar variáveis na Vercel, fazer redeploy.
- Se uma chave secreta for exposta, gerar uma nova no Supabase, atualizar Vercel e revogar a antiga.

3. Fluxo de alteração

- 1 Alterar os ficheiros-fonte locais; não editar `public` diretamente.
- 2 Executar `npm run build` na raiz.
- 3 Testar as páginas afetadas.
- 4 Rever `git status` e `git diff` para confirmar exatamente o que mudou.
- 5 Fazer `commit` com mensagem clara.
- 6 Fazer `push` para main.
- 7 Verificar o deployment automático na Vercel.

4. Arquitetura e publicação do ramo Sócios

O módulo é mantido em `portal/modules/socios`, transformado pelo build e publicado em `public/area/socios`. A base de dados e autenticação permanecem no Supabase.



`public/` é uma saída gerada. Uma correção feita apenas nessa pasta desaparece no build seguinte.

5. Ficheiros que deve conhecer

Ficheiro ou pasta	Responsabilidade
<code>portal/modules/socios/index.html</code>	Estrutura da página, diálogos, campos, botões, navegação e ligações para os PDFs.
<code>portal/modules/socios/app.js</code>	Estado, traduções, permissões, CRUD de sócios, quotas, filtros, histórico, CSV, validação e eventos.
<code>portal/modules/socios/styles.css</code>	Layout, cores, responsividade, tema escuro, tabelas, formulários e diálogos.
<code>portal/modules/socios/central-socios-client.js</code>	Cliente usado na variante autónoma; a Central publicada injeta configuração e autenticação partilhadas no build.
<code>portal/modules/socios/supabase/schema.sql</code>	Tabelas, funções privadas, triggers, constraints, grants e políticas RLS.
<code>portal/modules/socios/supabase/*.sql</code>	Migrações incrementais e reforços aplicados a bases já existentes.
<code>portal/modules/socios/docs/</code>	PDFs do utilizador e programador abertos pela janela Manuais.
<code>scripts/generate-manual-pdfs.py</code>	Fonte reproduzível dos PDFs publicados.
<code>scripts/prepare-vercel-output.mjs</code>	Copia módulos para public, injeta configuração central, versões de assets e páginas publicadas.
<code>vercel.json</code>	Build, output, headers de segurança, rewrites e rotas da Vercel.

6. Fluxo de execução no browser

- A página carrega a biblioteca Supabase, `central-config.js`, `config.js`, `central-module-auth.js` e `app.js`.
- `central-module-auth.js` confirma sessão e permissões globais antes de libertar a interface.
- `app.js` cria o cliente com `SUPABASE_URL` e a chave `anon/publishable`, nunca com `service_role`.
- As consultas a `members` e `member_audit_log` são autorizadas novamente pelo PostgreSQL através de RLS.

- A interface mostra ou esconde botões conforme as permissões, mas a segurança real deve permanecer na base de dados.
- O build remove dependências antigas da variante autónoma e injeta os scripts partilhados da Central.

7. Dados, auditoria e permissões

Objeto	Conteúdo e regra principal
public.members	Ficha administrativa: número, ata, adesão, quota, pagamento, identificação, contactos, profissão e observações.
public.member_audit_log	Registo imutável das inserções, edições e eliminações criado por trigger.
public.app_users	Perfil global, estado ativo e JSON de permissões por área/ação.
private.current_app_permission	Resolve a permissão efetiva da conta autenticada.
Triggers de members	Preenchem created_by/updated_by/updated_at e escrevem auditoria.
RLS	view autoriza SELECT; edit autoriza INSERT/UPDATE; delete autoriza DELETE; view_history autoriza auditoria.

Adicionar um botão no frontend sem criar ou rever a política RLS correspondente não é uma implementação completa.

8. Regras de validação que não devem ser removidas

- member_number é único quando preenchido.
- tax_number aceita nove algarismos quando existe.
- postal_code segue o formato 0000-000 quando existe.
- birth_date e quota_paid_at não podem ficar no futuro.
- A última quota paga não deve ser anterior à data de adesão.
- O frontend valida para dar mensagens úteis; constraints e RLS protegem a base mesmo perante pedidos manuais.
- A escrita de observações deve continuar separada da apresentação pública e da exportação quando a permissão não a permitir.

9. Como alterar a interface manualmente

- 1 Abrir a raiz do repositório num editor como VS Code.
- 2 Localizar o elemento em index.html, a lógica ou tradução em app.js e o aspeto em styles.css.
- 3 Pesquisar pelo id, classe, texto ou data-action do botão antes de alterar.
- 4 Fazer uma mudança pequena de cada vez e guardar com Ctrl+S.
- 5 Não alterar ids usados por document.querySelector sem atualizar todas as referências.
- 6 Quando adicionar texto visível, criar a versão portuguesa e inglesa na tabela de traduções.
- 7 Quando adicionar um campo, atualizar HTML, leitura do formulário, normalização, payload Supabase, edição, validação, histórico e exportação.

```
# Procurar um botão, campo ou tradução
rg -n "newMemberBtn|data-action|quotaPaidUntil" portal/modules/socios
```

```
# Rever apenas o módulo de Sócios
git diff -- portal/modules/socios scripts/generate-manual-pdfs.py
```

Evite substituições globais sem rever o resultado. Nomes de campos aparecem em HTML, JavaScript, SQL, importação, exportação e histórico.

10. Como adicionar ou alterar campos

Camada	Alteração necessária
PostgreSQL	Criar SQL incremental com ADD COLUMN IF NOT EXISTS, constraint adequada e eventual índice.
RLS e grants	Confirmar quem pode ler, criar, editar, eliminar ou exportar o novo dado.
index.html	Adicionar label, input/select/textarea, ajuda, id e name estáveis.
app.js - formulário	Adicionar ao FORM_FIELDS, leitura, limpeza, validação, preenchimento de edição e tradução.
app.js - persistência	Mapear entre camelCase e snake_case nos payloads e resultados.
Lista e consulta	Mostrar apenas se for útil e permitido; evitar expor identificadores desnecessários.
CSV/importação	Atualizar cabeçalhos, aliases, datas, validação e compatibilidade com ficheiros antigos.
Auditoria	Confirmar que o trigger inclui o campo e que o histórico o apresenta de forma compreensível.
Manual e testes	Documentar finalidade, cuidados e comportamento; testar criação, leitura, edição e permissões.

11. Guardar e rever alterações com Git

Guardar no editor escreve o ficheiro no disco. Git permite confirmar o que mudou e criar um ponto recuperável antes da publicação.

- Nunca usar git reset --hard para resolver uma dúvida num repositório com trabalho não publicado.
- Não incluir .env, chaves, exports, bases locais, anexos reais ou prints com dados pessoais.
- Se existirem alterações de outras pessoas, adicionar apenas os ficheiros da tarefa e rever o diff antes do commit.
- Mensagens de commit devem explicar a intenção, não apenas dizer update.

```
git status --short
git diff -- portal/modules/socios scripts/generate-manual-pdfs.py
git add portal/modules/socios scripts/generate-manual-pdfs.py
git commit -m "Atualiza area de socios e manuais"
git push origin main
```

12. Gerar os manuais e o site

- O primeiro comando atualiza apenas os dois PDFs de Sócios.
- npm run build compila Cibersegurança e gera toda a pasta public usada pela Vercel.
- O build deve terminar com exit code 0; avisos devem ser lidos, mesmo quando não impedem a geração.
- Confirme que public/area/socios/docs contém os dois PDFs novos depois do build.
- Se app.js ou styles.css mudou e o browser continuar a mostrar a versão antiga, atualize assetVersion em prepare-vercel-output.mjs e volte a gerar.

```
python scripts/generate-manual-pdfs.py --only socios
npm run build
```

13. Testar localmente antes de publicar

- Testar desktop e telemóvel, tema claro e escuro e, quando aplicável, português e inglês.

- Testar com uma conta que apenas vê e outra que edita; botões sem permissão não devem aparecer nem funcionar por pedido direto.
- Criar um registo fictício, consultar, editar, pagar quota e apagar apenas no ambiente de teste autorizado.
- Abrir Histórico e Manuais e confirmar que os PDFs carregam noutra separador.
- Confirmar que CSV abre com acentos e colunas corretas e não contém campos indevidos.
- Verificar a consola do browser e a ausência de erros HTTP ou JavaScript.

```
python -m http.server 4177 --directory public
# Abrir: http://127.0.0.1:4177/area/socios/
```

Não use dados reais em testes locais, screenshots ou vídeos de demonstração.

14. Publicar na Vercel

O caminho preferido é fazer push para main e deixar a integração Git da Vercel criar o deployment. A publicação manual é útil apenas quando o projeto local está corretamente ligado e a pessoa tem autorização.

- 1 Confirmar npm run build localmente.
- 2 Confirmar que o commit contém apenas os ficheiros pretendidos.
- 3 Fazer push para main ou executar a publicação manual autorizada.
- 4 Esperar pelo estado Ready na Vercel.
- 5 Abrir <https://central-mente-movimento.vercel.app/area/socios/> numa janela limpa.
- 6 Repetir o teste principal e abrir os dois PDFs em produção.

```
# Publicação automática
git push origin main
```

```
# Alternativa manual, depois de autenticar a CLI
npx vercel login
npx vercel --prod --yes
```

Tokens Vercel são secretos. Nunca os colocar no código, nos manuais, em commits ou em mensagens públicas. Se forem expostos, revogar imediatamente.

15. Alterações SQL e Supabase

- Nunca escolher Run without RLS para tabelas com dados de sócios.
 - Não usar DROP ... CASCADE em produção sem analisar dependências e ter plano de recuperação.
 - A service_role fica apenas em funções backend/variáveis protegidas; nunca no browser.
- 1 Criar um ficheiro SQL incremental na pasta supabase; não substituir dados manualmente no painel sem registo.
 - 2 Fazer backup e testar o SQL num projeto de teste ou com dados fictícios.
 - 3 Usar IF NOT EXISTS quando a operação puder ser repetida com segurança.
 - 4 Aplicar no SQL Editor com uma conta autorizada e ler todos os avisos de RLS.
 - 5 Recarregar o schema cache quando necessário e testar SELECT/INSERT/UPDATE/DELETE com perfis diferentes.
 - 6 Atualizar schema.sql para que uma instalação nova já inclua o estado final.

16. Reverter e recuperar

Problema	Ação recomendada
Deployment com erro	Promover o deployment anterior na Vercel ou criar um git revert do commit e voltar a publicar.

Problema	Ação recomendada
Erro apenas de interface	Corrigir a fonte em portal/modules/socios, gerar, testar e publicar; não corrigir só em public/.
Migração SQL incompleta	Parar novas escritas, avaliar dados afetados e aplicar uma migração corretiva testada; não improvisar DROP.
Dados apagados	Preservar auditoria, interromper ações adicionais e restaurar a partir do backup de acordo com o procedimento interno.
Chave exposta	Revogar no Supabase/Vercel, criar nova chave, atualizar variáveis e redeployar.

17. Checklist técnica final

- O código-fonte foi alterado, guardado e revisto com git diff.
- Os dois manuais de Sócios foram regenerados quando houve alteração funcional.
- npm run build terminou sem erro e public contém a versão esperada.
- CRUD, quotas, pesquisa, filtros, CSV, histórico, manuais e permissões foram testados.
- Não existem dados reais, segredos, .env, exports ou anexos no commit.
- O deployment está Ready e a URL de produção foi testada em desktop e telemóvel.
- Existe um caminho claro de rollback para código e base de dados.